

CSC422 Fall 2024 – Project Final Report

David Sweasey, Marilyn Thomas, Meer Sodhi

dbsease, mthoma26, mssodhi

dbsease@ncsu.edu, mthoma26@ncsu.edu, mssodhi@ncsu.edu

1 Chosen Project

Title: Skin Cancer Image Detection

Category: Predictive Modeling Projects

Dataset: Melanoma Skin Cancer Dataset of 10000 Images

<https://www.kaggle.com/datasets/hasnainjaved/melanoma-skin-cancer-dataset-of-10000-images>

Difficulty: * * * *

2 Abstract

Distinguishing between benign and malignant skin cancer from a large series of images can be time consuming and inefficient, especially with a large set of images. The primary objective of this project is to create a deep learning model that can accurately identify between benign and malignant melanoma. Such a model can provide timely diagnoses which may help save money and even lives. We will use a dataset containing 10,000 color images of skin cancer, and allow the model to train and validate on these images, ideally creating a model that can make predictions with sufficient accuracy. Our deep learning model will brandish the power of a convolutional neural network constructed using PyTorch and Google Colab.

In addition to developing an accurate predictive model, we hope to gain powerful insights into the capabilities of neural networks and predictive models. Specifically, we want to learn the potential for predictive models and their use in medicine. We yearn to gain confidence in the field of machine learning and in the various powerful libraries, such as PyTorch. Finally, we aim to explore model evaluation techniques in order to increase accuracy, prevent overfitting, and reduce cost.

Using our predictive models, we aim to discover what patterns exist in both images depicting benign or malignant tumors. We hope that if we can extract this information from our model, we can get a better understanding of how to diagnose types of skin cancer. Additionally, being able to extract this information unlocks many possibilities for predictive models in a wide array of fields.

3 Methodology

- We used *PyTorch* to create a functional convolutional neural network to predict whether an image of skin cancer is benign or malignant.
 - Enables simple construction of convolutional neural networks
 - Easy tuning of parameters
 - Efficient deployment of an accurate predictive model
 - Seamless preprocessing and data loading
- Google Colab will be used to write the code for, train, and validate our model.
 - Provides a power T4 GPU for fast training and testing
 - Allows for easy collaboration and code-sharing
- *PyTorch* provides much of the necessary functionality, but we will utilize other Python libraries as necessary, including:
 - NumPy
 - Pandas
 - Matplotlib
- Kaggle provides a dataset of 10,000 color images, all of which are 224x224.
 - Split training and testing sets. Already split into their classes: benign and malignant
 - We will need to create a PyTorch dataset object and assign each image with its appropriate class. This enables the neural network to seamlessly iterate over the entire dataset.
 - No, or very little, preprocessing will be required
- We begin with the model making its best guess on a batch of the training set. After each batch, we will calculate a loss and accuracy.
 - We will use *PyTorch* to back-propagate and adjust the weights of the neural network.
 - Continue until all training data has been seen by the model.
- Validate the model against the test set and calculate average accuracy and loss.
 - If 90% accuracy has not been achieved, we will adjust the model and its parameters and train again.
- We will analyze our model using heatmaps and confusion matrices and compare it to a decision tree
 - We want to see which is better for this binary classification task. We expect CNN to outperform a decision tree.

4 CNN

The Convolutional Neural Network (CNN) is a powerful tool for image classification that can extract various features and patterns from an image, and use these features to make a prediction.

We leveraged the power of a CNN to predict whether a given image depicts a benign or malignant skin lesion.

4.1 Preprocessing

CNNs require every image to be the same size and contain the same color channels. Color channels refers to how many values represent the color. Typically, this means either RGB or black and white. Thankfully, every image from the Kaggle dataset was color (RGB) and was sized at 224x224. There were no repeats nor extraneous data, and any outliers were kept because any real person could have an atypical skin lesion that still requires determination. For an accurate and realistic model, we must allow any and all atypical lesions, as these can occur in real life. Therefore, the only preprocessing we did was convert every raw image into a PyTorch specific tensor so our model can effectively use them.

4.2 DataLoaders

Kaggle also provided the data split into *train* and *test* datasets. The test dataset contains 1,000 images while the train dataset contains over 9,000. Seeing no issue with the two groups, we maintained this split, meaning we used the holdout method with a 90-10 split for validation.

We constructed PyTorch dataloaders: one for each dataset. These are objects which combine data objects (images in our case) into arbitrarily sized batches that PyTorch can then iterate over, batch by batch. Whether or not to shuffle the dataset can also be specified. This is important for training, for a model that trains on a dataset that is presented in the same order every time may begin to fit to the *order* of the data, rather than the *content*. The following is a code snippet showing the construction of these objects.

```
# How many images to train on before updating weights and reporting accuracy and loss
batch_size = 16

# How many workers "load" data into the model for training simultaneously
num_workers = 2

# Create DataLoaders. True/False refers to shuffle
trainloader = D.DataLoader(train_set, batch_size, True, num_workers = num_workers)
testloader = D.DataLoader(test_set, batch_size, False, num_workers = num_workers)
```

The variable *num_workers* refers to how many processes are used to load data. Too many causes significant CPU and memory overhead, and too few causes the model to train slower because the GPU is waiting on the data to be loaded. As mentioned above, the GPU we are using is the remote compute unit T4 GPU provided by Google Colab.

4.3 Model Choice

Once loaded, we define our model. For initial training and testing, we constructed our own model of 5 convolutional layers, 5 pooling layers, and 5 fully connected layers. We then moved

on to a new-born ResNet18 model. The following results will be from this model. This model has many more convolutional layers, meaning it should be more effective in extracting features and patterns. Additionally, it performs downsampling less frequently. Although this makes training slower, we retain more of the data deeper through the neural network. ResNet18 has only a single average pooling layer at the end followed by only one fully connected output layer.

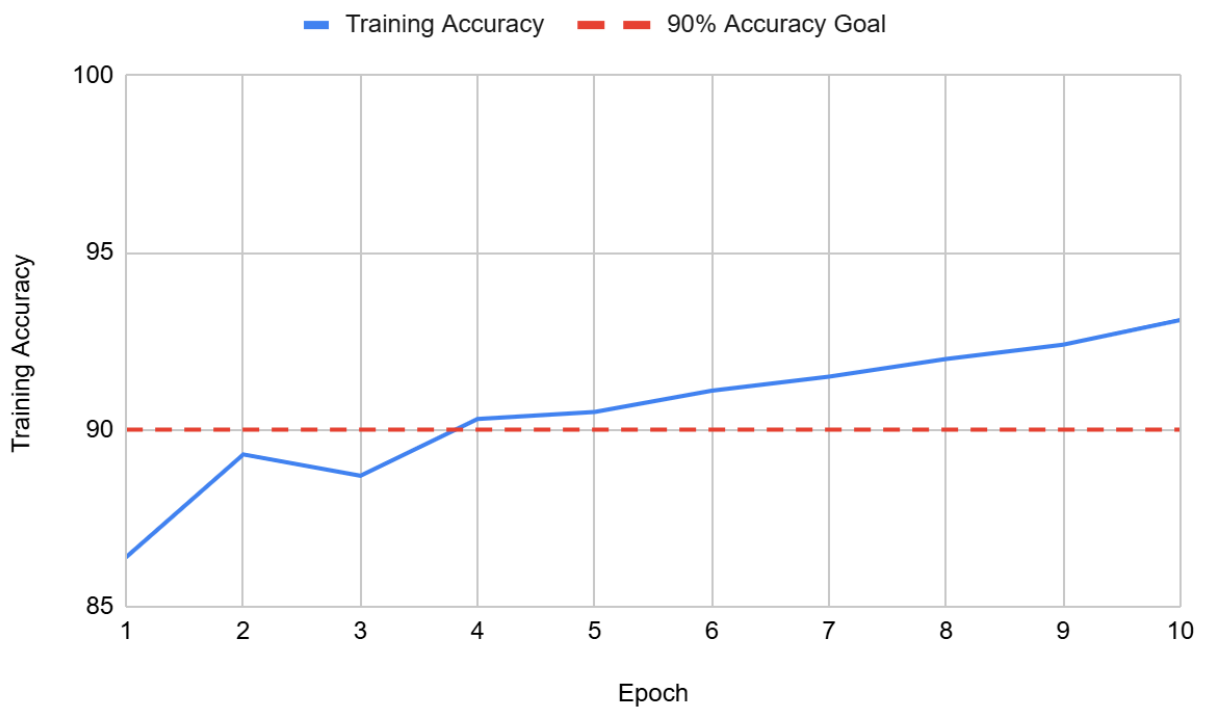
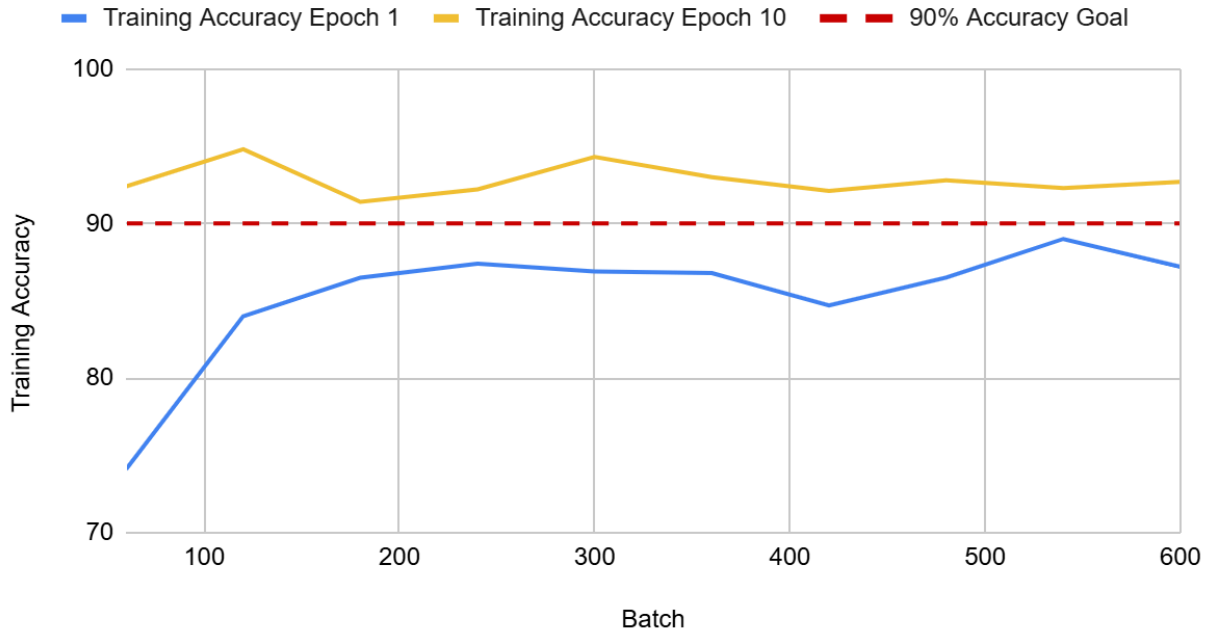
Because of the better feature extraction and since it is a known and proven model for image classification, we chose ResNet18. Additionally, many of the convolutional layers require the same number of input channels as they output. Although this is not revolutionary, it does help when generating visualizations such as heatmaps, since a large number of output channels means a large number of heatmaps.

Finally, we found that a learning rate of 0.0001 was most effective for quickly training. The training accuracy increased significantly slower with a smaller rate, and experienced too many fluctuations with a larger rate. Additionally, we used *cross entropy loss* as the loss function, and *Adam* as the optimizer. These are widely used in image classification, and served us well as seen by the results below.

4.4 Training

The model was trained over the same training dataset for 10 epochs. After every 60 batches, the accuracy and loss were reported until the entire dataset was exhausted. The training accuracy slowly increases from 50% (a pure guess) to a maximum of 94.8%. It must be noted that training accuracy is not as important as testing accuracy because of issues like overfitting and lack of generality. However, it is an important metric to gauge how effectively the model is learning. The following depicts the training accuracies over the first and 10th epoch, as well as the average training accuracy per epoch.

Training Accuracies per Batch for Epochs 1 and 10



Not pictured on the first graph is the accuracy after the 0th batch. The program I wrote did not report this, because it is trivial. The 1st epoch 0th batch accuracy is 50%, since it is guessing between 0 (benign) and 1 (malignant). Any successive epoch's 0th batch accuracy is just the final

accuracy of the previous epoch. After the initial spike from 50% to over 80% from the 0th to the 60th batch, the accuracies tend upward gradually, with various fluctuations as the model continues to learn. We can see by the 4th epoch in the second graph that we reach our goal of 90% accuracy. Again, this is training accuracy so not nearly as important as testing accuracy, but it provides a good reality check as to where the model is.

4.5 Testing

The model was validated over the test dataset, containing 1,000 unique images. The model did not train on these, so upon each testing run, it has never seen these images. If it had trained on these images, we would get no insight as to how its performance may be on a completely new image. As mentioned previously, testing reveals the true accuracy of a model as well as how susceptible it is to overfitting.

After each epoch of training, we tested the model. Importantly, we included the following lines of code to ensure the model was not affected by testing.

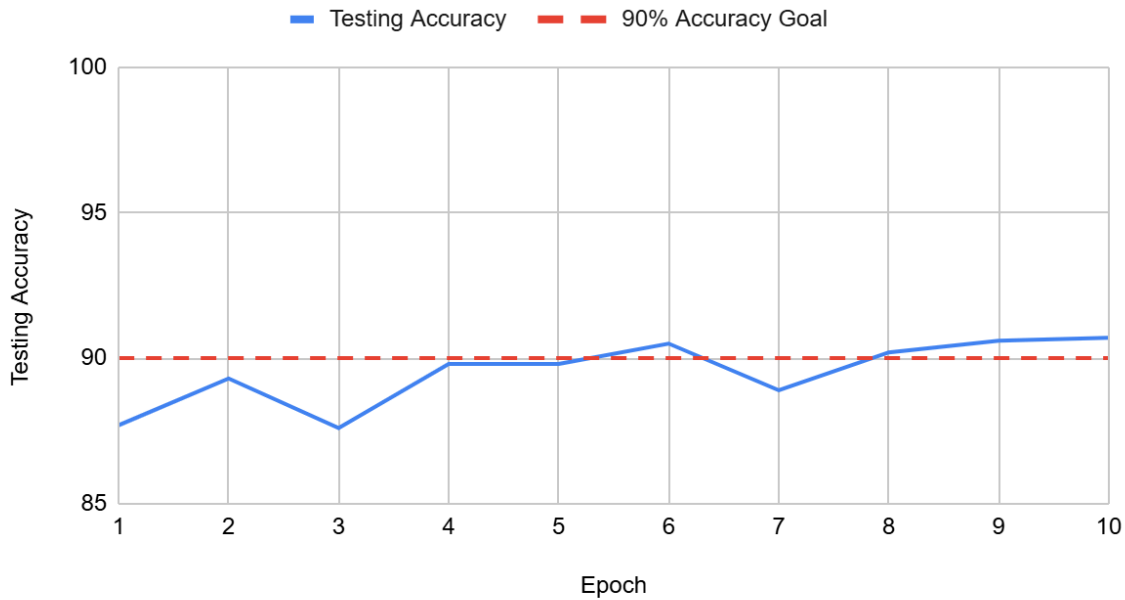
```
net.train(False)
```

```
with torch.no_grad():
```

Otherwise, the *test_epoch()* function operates the same as the training function: it iterates over the testloader object, passes a batch of images into the model, computes the loss and accuracy, and continues as such. Additionally, there is an option to generate a confusion matrix, which runs only on the last epoch (where the model ideally is at its best).

Below is a graph displaying the testing accuracy after each epoch of training. There are gradual fluctuations, but the testing accuracy displays an upwards trend peaking at 90.7% at the end of the 10th epoch.

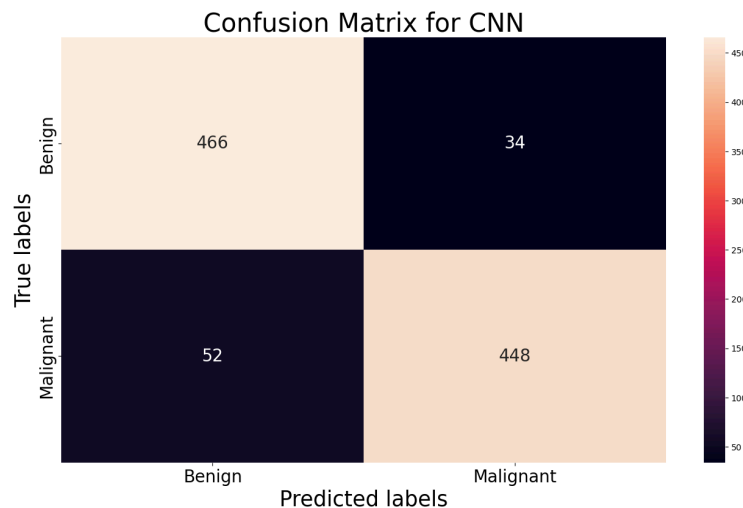
Testing Accuracy per Epoch



The model after the 10th epoch is the most accurate, and thus the model we care about. We have a **testing accuracy** of 90.7% and an average **training accuracy** of 92.8%.

4.6 Visualization

After all of the training and testing, we constructed some visualization methods to gain insight into how our model works and was performing. The first is a confusion matrix. The matrix below was generated on the test dataset after the 10th epoch of training.

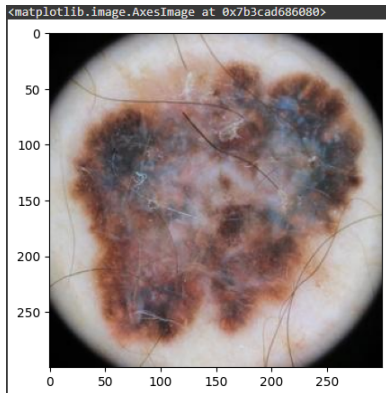


Based on the confusion matrix, the model tends towards predicting benign tumors. 51.8% of predictions are benign, while 48.2% of images are predicted to be malignant. Calculating precision, recall, and F1 yields:

- **Precision: 0.9295**

- **Recall: 0.896**
- **F1 Score: 0.9124**

The second visualization metric we used were heatmaps. Essentially, we ran an image through 5 of the convolutional layers to extract the feature maps. Using ResNet18, 5 layers produce 64 feature maps. Then we ran the image through the entire CNN to yield a prediction. These feature maps give an idea of what the model “sees” and how it makes predictions. Interestingly, every significant feature of the below image is seen by the model. Below is the tumor we generated heatmaps for.

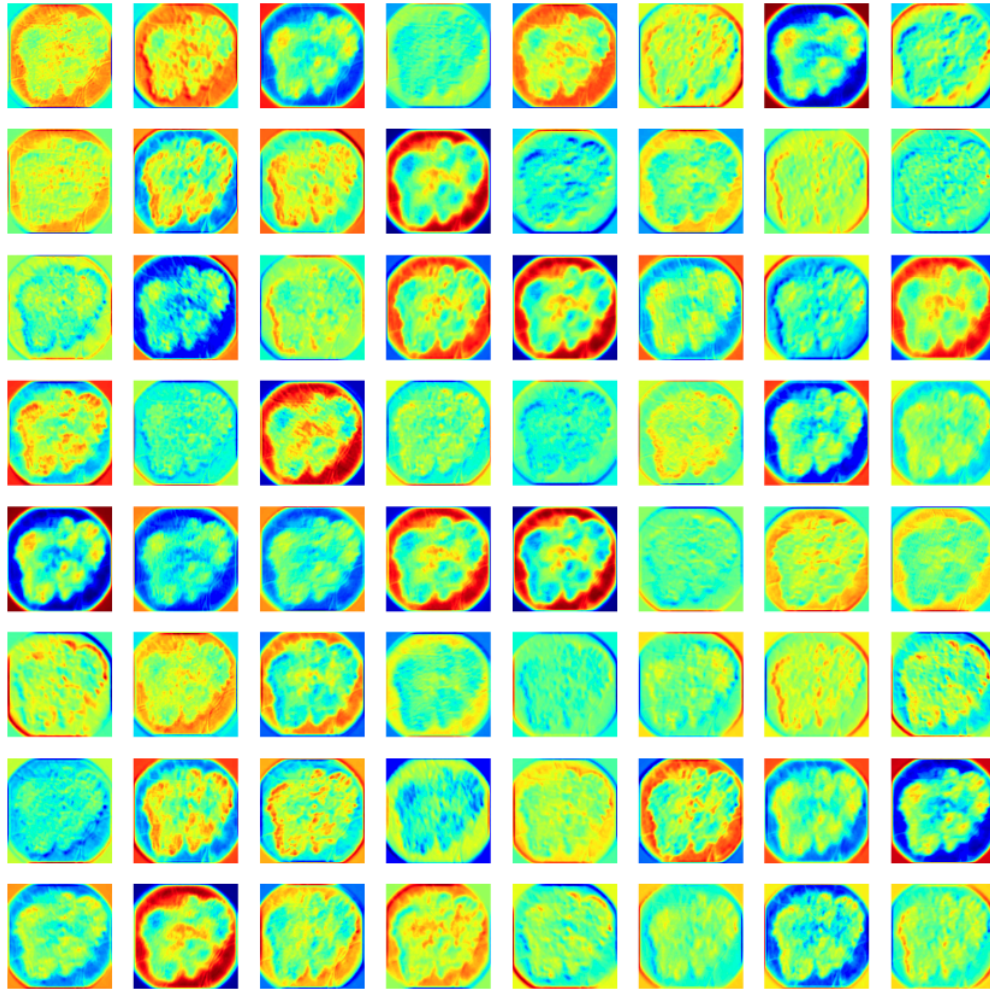


Running the following code generates the following 64 heatmaps. This code passes an image through 5 convolutional layers. The original image is passed through the first layer, and then those results (as a CNN normally works) are passed through the second layer, and so on.

```
# Shows the feature maps after 5 convolutional layers (could change this)
results = [conv_layers[0](image)]
for i in range(1, 5):
    results.append(conv_layers[i](results[-1]))

layer_visualize = results[-1].squeeze()
layer_visualize = layer_visualize.detach().cpu().numpy()
plt.figure(figsize=(15,15))
plt.set_cmap('jet')
for i in range(layer_visualize.shape[0]):
    plt.subplot(8, 8, i+1)
    plt.imshow(layer_visualize[i])
    plt.axis('off')
```

The all 64 channels of the final result are then displayed using Matplotlib, and using the *jet* colormap results in a nice looking heatmap. The original photo can be seen in most of these with a color shift. We can see that several focus on the skin outside of the tumor, but even more focus on the darker spots of the skin lesion.



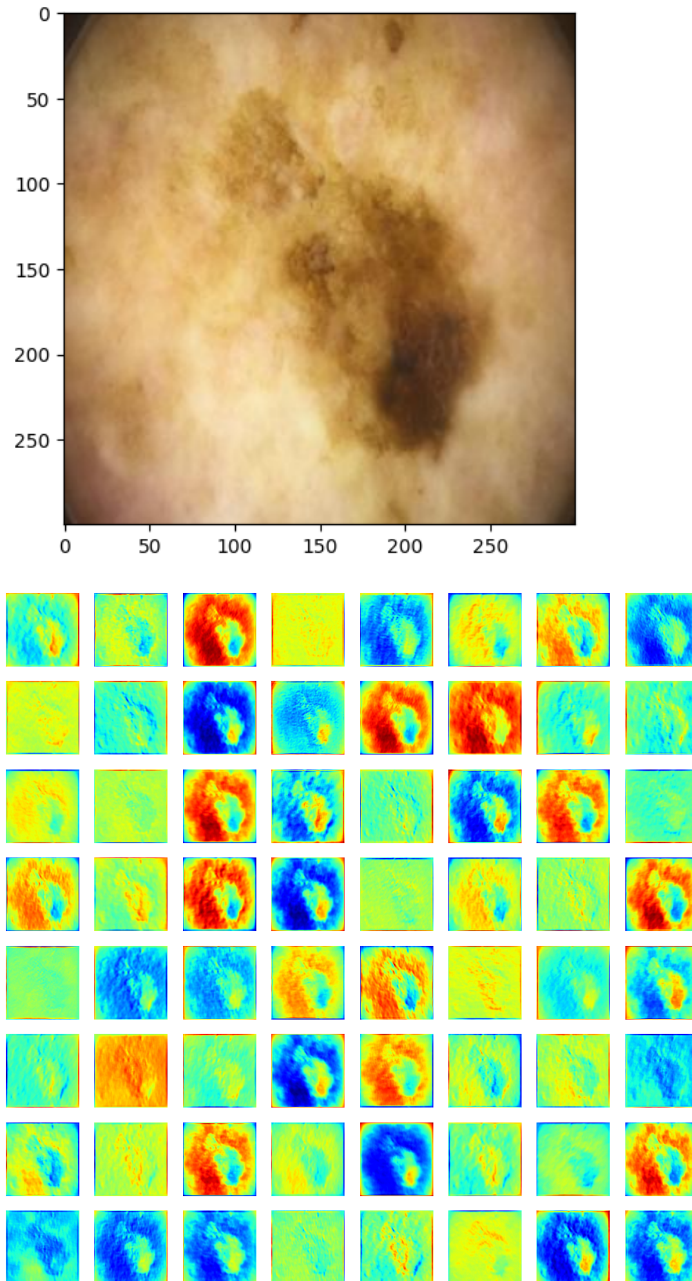
4.7 Discussion

Overall, CNN performed slightly better than our expectations. We understood that CNNs are great for image classification, but for a task where sometimes it can be very difficult to differentiate images, we were impressed. We surpassed our original goal of 90% training error.

Reviewing the evaluation metrics of precision, recall, and F1 we see a very well performing model. However, the recall score is a slight concern. In medicine, the ideal outcome of a diagnosis is a correct one. So, a high accuracy (90.7%) is important for a model. Furthermore, a patient would likely prefer to be diagnosed with something they are not afflicted with rather than miss a potentially life-threatening diagnosis. Again, both are quite important, but the latter may be more endangering if wrong. Therefore, **recall** should have a slight prioritization with our model. However, it was the lowest of the three metrics (accuracy, precision, recall). Though not by much, this illuminates that our model prefers making benign predictions.

Reviewing the heatmap, we can see several images where the model scans extraneous information, such as the black background. However, because there are 64 (and eventually 512

with ResNet18) feature maps, this is likely not an issue. The model can clearly define the shape of the skin lesion and the unusually dark spots on the lesion. With several benign lesions, the shape is relatively circular and the color is consistent throughout. However, oftentimes with a malignant tumor, the shape is less circular and more sporadic and has an inconsistent color, often with many dark splotches within. The model clearly picks up on this detail. However, we analyzed the feature maps of a malignant tumor that the model classified as benign.



The model has an interesting fixation with the darker spot in the middle of the lesion as well as the surrounding area. However, from this it does not conclude that this is indeed cancerous. It is hard to know for sure why, but we speculate that because the surrounding area on which the

model fixates is relatively normal, the model assumes that the lesion must be benign, despite the focus on the darker spot. Additionally, the picture appears a little blurry and slightly discolored which could easily throw the model off.

Overall, the CNN performed very well, and with some additional hyperparameter tuning, some medical professionals on board, additional training, and a more diverse dataset, we believe this model could be utilized by the medical field. It was impressive to see what kinds of patterns the model picks up on, and many are patterns already commonly recognized by epidemiological professionals.

5 Decision Tree

We were certain that a CNN was the best option for a skin cancer predictive model because they are specifically designed to recognize spatial hierarchies and patterns, automatically extracting features without manual engineering. Unlike traditional machine learning or non-convolutional models, CNNs handle variations in size, angle, and lighting effectively, making them uniquely suited for analyzing complex medical images like skin lesions. However, we were interested in challenging our CNN by exploring other potentially successful models. To achieve this, we chose to implement a decision tree for a skin cancer predictive model.

5.1 Preprocessing

The input images are normalized and resized to a uniform dimension (224×224 pixels).

- **Normalization** scales pixel values to $[0,1]$, improving numerical stability during training and testing.
- **Resizing** ensures that all images have consistent dimensions for processing. In this case, all images were resized to a uniform dimension of 224×224 pixels to ensure consistency.

Feature Extraction:

The images are flattened into 2D arrays so the decision tree can process them. Each pixel in the image, across three color channels (R, G, B), becomes a separate feature. For a 224×224 image with 3 channels, this results in 150,528 features per image ($224 \times 224 \times 3$).

Since Decision Trees lack inherent processes to identify hierarchical spatial relationships, each pixel served as an independent feature. This simple feature extraction approach controls the model's ability to capture complex patterns in the image data, which is critical for high-dimensional inputs like images.

5.2 Model Training

The Decision Tree Classifier is trained using:

- **Criterion:** Gini impurity to decide splits, ensuring the most significant reduction in impurity at each node.
- **Max Depth:** Limited to 10 to prevent overfitting by restricting the tree's complexity.
- **Min Samples Split:** Minimum number of samples required to split an internal node.
- **Min Samples Leaf:** Minimum number of samples required to be at a leaf node.

5.3 Model Validation and Testing

The trained model is evaluated on unseen validation and test datasets to measure its generalization performance.

Performance Metrics:

To produce the best results, we were focused on tuning the tree's hyperparameters. Initially, we attempted to use *GridSearchCV* to automatically and iteratively test the best hyperparameters for the decision tree based on scoring. However, *GridSearchCV* proved to be too computationally expensive for the machine to handle. Then, we attempted to use other algorithms that could determine the best hyperparameters, but we ended up with the same issue as *GridSearchCV*. As a result, we determined the best hyperparameters through trial and error. In determining the best hyperparameters, we prioritized adjusting `max_depth`, `min_samples_split`, and `min_samples_leaf`. We found that a decision tree with a max depth of 10 was the most optimal despite slight overfitting.

Exact Accuracy Numbers:

Training Accuracy: 0.9573138990109318

Validation Accuracy: 0.8672566371681416

Test Accuracy: 0.885

The drop in accuracy from training to validation/testing indicates potential overfitting.

1. **Classification Report:** Precision, recall, and F1-scores average approximately 0.89 across classes:
 - **Precision:** Measures how many predicted malignant cases are correct.
 - **Recall:** Measures how many actual malignant cases are detected.
 - **F1-Score:** Balances precision and recall, useful for imbalanced datasets.

```

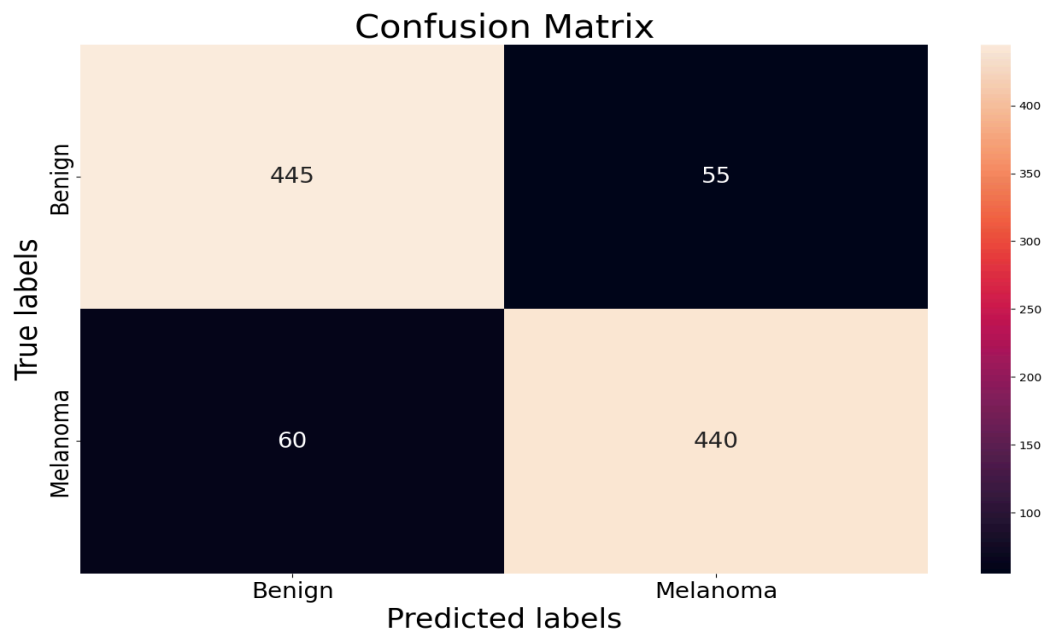
Classification Report:
              precision    recall  f1-score   support

     0.0         0.88      0.89      0.89         500
     1.0         0.89      0.88      0.88         500

 accuracy          0.89         1000
 macro avg         0.89      0.89      0.88         1000
 weighted avg      0.89      0.89      0.88         1000

Confusion Matrix:
[[445  55]
 [ 60 440]]

```



5.4 Strengths vs Weaknesses Decision Trees:

Strengths:

- High interpretability:
 - Decision Trees are easily visualized making them accessible for understanding and explaining model decisions.
- Baseline Performance:
 - Achieved a reasonable F1-score demonstrating its capability as a baseline model.

Weaknesses:

- Overfitting:
 - The gap between training and testing accuracies suggests limited generalized ability.
- Handling Large Feature Spaces

- The model struggles to process high-dimensional inputs like images without significant pruning or preprocessing.
- Limited Scalability:
 - Performance degrades for complex datasets where relationships among features are non-linear or hierarchical.
- Relatively low generalization capability compared to CNNs.

5.5 Comparison to CNN

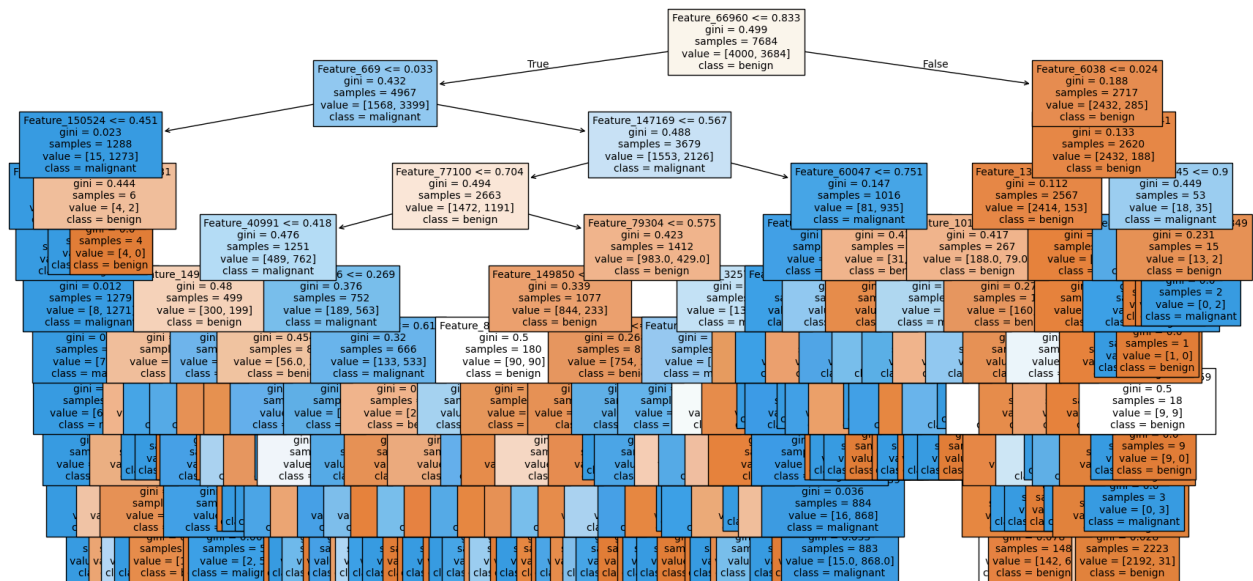
The evaluation metrics used (Test accuracy, Validation accuracy, Testing accuracy, Precision, Recall, and F1 score) revealed that the CNN had better performance.

CNNs outperformed the Decision Trees especially since the task at hand involved image data and their ability to capture spatial and hierarchical features surpass the Decision tree's abilities. While the Decision Tree achieved an F1-score of ~0.89, CNNs consistently demonstrated higher precision, recall, and generalization capability for the same dataset.

For a Direct Compare and Contrast, the CNNs score is on the right compared to the Decision Tree Score on the left.

	Decision Tree	CNN
Precision	~0.89	0.93
Recall	~0.89	0.90
F1	~0.89	0.91

The Decision Tree Classifier achieves a reasonable performance with an F1-score of ~0.89 for predicting skin cancer. However, overfitting and limited scalability to image data are significant drawbacks. While this model provides interpretability, transitioning to CNNs or ensemble methods is recommended for better real-world applicability.



5.6 Challenges:

- **High Dimensionality:** Processing 150,528 features (pixels) strained computational resources, limiting scalability.
- **Overfitting:** Despite limiting the tree depth, a performance gap between training and testing accuracies persisted.
- **Computational Restrictions:** Automated hyperparameter tuning tools like GridSearchCV were too resource-intensive.

5.7 Analytics and troubleshooting:

- Feature Preprocessing (normalization and resizing) was optimized to ensure numerical stability.
- Hyperparameters were tuned manually after automated methods proved infeasible.
- The use of max_depth, min_samples_split, and min_samples_leaf reduced overfitting but did not eliminate it entirely.

6 Contributions

Neural Network Implementation: David Sweasey

Decision Tree Implementation: Meer Sodhi

Evaluation and Comparison: All

Report Preparation: Merilyn Thomas